BMP388 Pipeline (I2C)

Introduction

The BMP388 is a pressures & temperature sensor developed by Bosch for drones, weather stations and other devices deriving height or weather conditions. Internally the BMP388 consist of a piezo resistive sensing element and a mixed signal ASIC, the ASIC performs A/D conversions and provides the conversion results and sensor specific compensation data through a digital interface.

For this pipeline the main goal is to get a absolute value which can be used as a anchor point by other sensors. This is key to what manner the data should be extracted since if were used for a purpose such as measuring sink/climb rate using fifo with a constant time base is best for derivatives but since this is a application where the time constant isn't needed the fifo wont be used and direct measurements from the read register will be read.

I2C interface

When SDO is Grounded the device slave address is 0x76, connecting to 3.3V give it slave address 0x77, The SDO pin cannot be left floating, the I2C address will be left undefined.

I2C writing

Writing to the device is done by 2 bytes where byte[0] = register to write to and byte[1] = register data. Each write consist of 2 bytes.

Example (write):

```
static void bmp_wr(uint8_t reg, uint8_t val) {
uint8_t buf[2] = {reg, val};
i2c_write_blocking(i2c1, BMP_ADDR, buf, 2, false);
}
```

I2C reading

Reading from the device consist of 2 operations one is a write to the register which needs to be read then the following state on the sensor is a read operation where it will dictate n ammount of words depending on the configuration, a typical I2C read function is as follows:

```
static void bmp_rd(uint8_t reg, uint8_t *buf, size_t n) {
i2c_write_blocking(i2c1, BMP_ADDR, &reg, 1, true);
i2c_read_blocking(i2c1, BMP_ADDR, buf, n, false);
}
```

Program init

Sampling rates & Resolution

OSR & ODR

The OSR is used to specify the output data rate & noise on this specific device, For the BMP388 at 1x “no over sampling” the noise is roughly ±1 Pa RMS class, which maps to roughly ±8–10 cm of altitude on raw noise samples at 40HZ. If sampled at x32 the conversion time scales at 2^n so each read takes a few hundred milliseconds but filtered. The tradeoff comes down to speed and resolution. The ODR gives the rates of which It sends out the samples, so 5ms = (1/0.005 = 200Hz) so take 5ms∗32 = 160 & 1/0.160 = 6.25hz. Ideally the ODR pre scaler should match the OSR setting.

Table 40: Register 0x1C "OSR"

Bits	Name	Value	Setting	Description
Bit 2..0	osr_p	000	x1	no oversampling
		001	x2	x2 oversampling
		010	x4	x4 oversampling
		011	x8	x8 oversampling
		100	x16	x16 oversampling
		101	x32	x32 oversampling
Bit 5..3	osr4_t	000	x1	no oversampling
		001	x2	x2 oversampling
		010	x4	x4 oversampling
		011	x8	x8 oversampling
		100	x16	x16 oversampling
		101	x32	x32 oversampling

Table 41: Register 0x1D "ODR"

The "ODR" register sets the configuration of the output data rates by means of setting the subdivision/subsampling.

Bits	Name	Description
Bit 4..0	odr_sel	Subdivision factor for pressure and temperature measurements is 2^value. Allowed values are 0..17. Other values are saturated at 17.

Table 42: Control Settings for odr_sel

odr_sel	Pre scaler	Name	Description	Sampling Period
0x00	1	ODR_200	ODR 200 Hz	5 ms
0x01	2	ODR_100	ODR 100 Hz	10 ms
0x02	4	ODR_50	ODR 50 Hz	20 ms
0x03	8	ODR_25	ODR 25 Hz	40 ms
0x04	16	ODR_12p5	ODR 25/2 Hz	80 ms
0x05	32	ODR_6p25	ODR 25/4 Hz	160 ms
0x06	64	ODR_3p1	ODR 25/8 Hz	320 ms
0x07	128	ODR_1p5	ODR 25/16 Hz	640 ms
0x08	256	ODR_0p78	ODR 25/32 Hz	1.280 s
0x09	512	ODR_0p39	ODR 25/64 Hz	2.560 s
0x0A	1024	ODR_0p2	ODR 25/128 Hz	5.120 s
0x0B	2048	ODR_0p1	ODR 25/256 Hz	10.24 s
0x0C	4096	ODR_0p05	ODR 25/512 Hz	20.48 s
0x0D	8192	ODR_0p02	ODR 25/1024 Hz	40.96 s
0x0E	16384	ODR_0p01	ODR 25/2048 Hz	81.92 s

Calibration process

On the BMP388 the calibration process is done during the init sequence, Every Bosch die has manufacturing spread: the MEMS membrane's sensitivity, the ADC offset, the bandgap temperature sensor slope which very chip to chip. During production tests, each die is measured against reference pressure/temperature points, computes correction coefficients and laser trims one time programs into NVM. Theres 21 bytes at 0x31–0x45 that are the correction data "trimming coefficients" since Bosch NVM coefficients are only 8 or 16 bit wide, each coefficient is stored as a fixed point integer – a integer with a implicit binary point at a defined position. To convert stored values Bosch has a list of formulas published on the device datasheet.

For calibration the values (T1–T3) (P1–P11) & t_lin can be stored into a struct for the compensation during run time. During the init sequence the calibration process is only to get the values used for the pressure compensation on runtime.

Table 22: Trimming Coefficients (Memory Map)

Memory ADR \[dec]	Memory ADR	Register content	Trimming Coefficient	Size	Sign
69	0x45	NVM_PAR_P11<7:0>	NVM_PAR_P11	8 Bit	signed
68	0x44	NVM_PAR_P10<7:0>	NVM_PAR_P10	8 Bit	signed
67	0x43	NVM_PAR_P9<15:8>	NVM_PAR_P9	16 Bit	signed
66	0x42	NVM_PAR_P9<7:0>			
65	0x41	NVM_PAR_P8<7:0>	NVM_PAR_P8	8 Bit	signed
64	0x40	NVM_PAR_P7<7:0>	NVM_PAR_P7	8 Bit	signed
63	0x3F	NVM_PAR_P6<15:8>	NVM_PAR_P6	16 Bit	unsigned
62	0x3E	NVM_PAR_P6<7:0>			
61	0x3D	NVM_PAR_P5<15:8>	NVM_PAR_P5	16 Bit	unsigned
60	0x3C	NVM_PAR_P5<7:0>			
59	0x3B	NVM_PAR_P4<7:0>	NVM_PAR_P4	8 Bit	signed
58	0x3A	NVM_PAR_P3<7:0>	NVM_PAR_P3	8 Bit	signed
57	0x39	NVM_PAR_P2<15:8>	NVM_PAR_P2	16 Bit	signed
56	0x38	NVM_PAR_P2<7:0>			
55	0x37	NVM_PAR_P1<15:8>	NVM_PAR_P1	16 Bit	signed
54	0x36	NVM_PAR_P1<7:0>			
53	0x35	NVM_PAR_T3<7:0>	NVM_PAR_T3	8 Bit	signed
52	0x34	NVM_PAR_T2<15:8>	NVM_PAR_T2	16 Bit	unsigned
51	0x33	NVM_PAR_T2<7:0>			
50	0x32	NVM_PAR_T1<15:8>	NVM_PAR_T1	16 Bit	unsigned
49	0x31	NVM_PAR_T1<7:0>			

Calibration coefficient (9.1 – data sheet)

$$PAR_T1 = \frac{NVM_PAR_T1}{2^{-8}} \qquad PAR_T2 = \frac{NVM_PAR_T2}{2^{30}} \qquad PAR_T3 = \frac{NVM_PAR_T3}{2^{48}}$$

$$PAR_P1 = \frac{NVM_PAR_P1 - 2^{14}}{2^{20}} \qquad PAR_P2 = \frac{NVM_PAR_P2 - 2^{14}}{2^{29}} \qquad PAR_P3 = \frac{NVM_PAR_P3}{2^{32}}$$

$$PAR_P4 = \frac{NVM_PAR_P4}{2^{37}} \qquad PAR_P5 = \frac{NVM_PAR_P5}{2^{-3}} \qquad PAR_P6 = \frac{NVM_PAR_P6}{2^{6}}$$

$$PAR_P7 = \frac{NVM_PAR_P7}{2^{8}} \qquad PAR_P8 = \frac{NVM_PAR_P8}{2^{15}} \qquad PAR_P9 = \frac{NVM_PAR_P9}{2^{48}}$$

$$PAR_P10 = \frac{NVM_PAR_P10}{2^{48}} \qquad PAR_P11 = \frac{NVM_PAR_P11}{2^{65}}$$

Table 39: Register 0x1B "PWR_CTRL"

PWR_CTRl is used to set the mode which the device will be operating on.

Bits	Name	Value	Setting	Description
Bit 0	press_en	0	press_off	Disables the pressure sensor.
		1	press_on	Enables the pressure sensor.
Bit 1	temp_en	0	temp_off	Disables the temperature sensor.
		1	temp_on	Enables the temperature sensor.
Bit 5..4	mode	00	sleep mode	Sleep mode
		01/10	forced mode	Forced mode
		11	normal mode	Normal mode

Table 24: Register 0x00 "CHIP_ID"

Register 0x00 is the who am I read register which is used to check if machine is connected. Used for verification.

Bits	Name	Description
Bit 7..0	–	Chip id.

BMP read

Reading the BMP388 consist of reading 6 byes of data and parsing it into 32 bit values and using the compensation formulas listed on 9.2 on the data sheet.

Table 27: Pressure Data Registers

The 24-bit pressure data is split and stored in three consecutive registers:

0x04 "DATA_0"
0x05 "DATA_1"
0x06 "DATA_2"

	0x06 DATA_2	**0x05 DATA_1**	**0x04 DATA_0**
Register	0x06	0x05	0x04
Bit	7..0	7..0	7..0
Name	PRESS_MSB_23_16	PRESS_LSB_15_8	PRESS_XLSB_7_0
Data	pressure data		

Temperature & pressure compensation

```
uint8_t d[6];
bmp_rd(REG_DATA, d, 6);
uint32_t up = ((uint32_t)d[2] << 16) | (d[1] << 8) | d[0];
uint32_t ut = ((uint32_t)d[5] << 16) | (d[4] << 8) | d[3];

// Data sheet 9.2 Temperature

double pd1 = (double)ut - c->T1;
double t_lin = pd1 * c->T2 + pd1 * pd1 * c->T3;
c->t_lin = t_lin;
*temp_c = t_lin;

// Data sheet 9.3 Pressure

double t2 = t_lin * t_lin, t3 = t2 * t_lin;
double o1 = c->P5 + c->P6*t_lin + c->P7*t2 + c->P8*t3;
double o2 = (double)up * (c->P1 + c->P2*t_lin + c->P3*t2 + c->P4*t3);
double up2 = (double)up * up;
double o4 = up2 * (c->P9 + c->P10*t_lin) + up2 * (double)up * c->P11;
*press_pa = o1 + o2 + o4;
```

```
BMP_ADDR      0x76
REG_CHIP_ID   0x00
REG_DATA      0x04
REG_PWR_CTRL  0x1B
REG_OSR       0x1C
REG_CALIB     0x31
```